Upgrading Daewoo CPC-400/400S into the standard MSX2/MSX2+ computer

Document revision: 1.01

Wierzbowsky [RBSC]

April 16, 2026

This document describes the process of converting the Korean Daewoo CPC-400/400S computer into a fully functional and more compatible computer with a Japanese-Russian BIOS. The document describes not only the replacement of the MSX2 ROMs with more compatible ones, but also the upgrade of this computer to a full-fledged MSX2+ with an F4 port, an FMPAC Light sound card, and two megabytes of external RAM.

Table of Contents

1. Acknowledgments 3

2. Introduction 3

3. Project Goals 4

4. Analysis and Initial Modification 5

5. Initial Challenges and Their Solutions 7

6. External Modules 7

7. First VHDL Firmware 8

8. New Problems—Disk ROM 8

9. The "self-destruction" of the Disk ROM 9

10. New Board Versions 9

11. Per Aspera Ad Astra. And Beyond 10

12. MSX2+ is Always Better than MSX2 10

13. How to Replicate the Project 11

14. New Slot Configurations 12

15. IMPORTANT INFORMATION! 16

16. Project Repository 17

17. Contact Information 17

This document is not an official guide, and anyone who decides to replicate this upgrade does so **at their own risk**. The author is not responsible for any issues that may arise from third parties upgrading the computer. There may be some inaccuracies in the guide, so if you notice any, please contact the author. Contact information is provided at the end of this document.

Detailed step-by-step instructions on what needs to be changed on the Daewoo motherboard and which files need to be written to the EPROM or programmable logic chips are provided in a separate file to make the upgrade process easier. The file is named "**Daewoo_CPC400_Upgrade_HowTo.pdf**" and is located in the "Docs" folder of the repository.

1. Acknowledgments

So, where to begin describing this serious and lengthy project? I'll start with acknowledgments. I'd especially like to highlight **Maxim Vlasov (Meteor-M)**. Without him, this project would hardly have come to fruition. Maxim helped me throughout the entire project, but his assistance was especially valuable at the very beginning—when I needed to understand how programmable logic chips work. Later on, Maxim helped with advice on hardware, CPLD firmware, and BIOS. Thank you to him for his immense patience and vast knowledge of MSX topics! I'd also like to thank **the members of the RBSC** (Russian Bear Service Crew) for their comprehensive support and advice. I'd also like to thank the developers of the openMSX emulator for the excellent debugger, which was a huge help in adapting the disk ROM and resolving other issues.

2. The Beginning

Well then, let's start with the history of this project. And this story, strange as it may seem, is directly linked to the Carnivore2 multi-functional cartridge developed by the RBSC group. When we were writing the startup menu for our cartridge, we unexpectedly ran into compatibility issues on Korean and Arabic MSX computers. These computers display not only Latin characters but also their own special symbols (hieroglyphs), which usually don't fit into the standard 8x8-pixel character set. As a result, the display is achieved by combining different characters into a single large 16x16 block. Because of this, the display logic in text modes changes. Additionally, extra space is reserved in the fonts data for these characters. And to support this text display mode, special language modules are loaded from ROM when the system starts. In the Daewoo 400 and 400S, this module is called "Hangul." And it was precisely the operation of this module that prevented our boot menu from correctly displaying characters for the interface. But that wasn't all. The fact is that the Hangul module apparently contains some errors in its code. We discovered that the Z80 processor's shadow registers become corrupted upon returning from a call to this module. As a result, some of our subroutines received corrupted data, and the interface stopped working. Since removing this module was not possible at the time, a special version of the startup menu was created for Korean and Arabic computers. Ever since then (and by 2026, 10 years had already passed), I couldn't stop thinking about making the Daewoo CPC-400 and 400S more similar to the Japanese MSX2 models.

The second problem we encountered while debugging Carnivore2 was that quite a few games and other software on Korean computers refused to run due to the rather unusual placement of the BIOS modules. In standard MSX2 computers, the MSX BIOS and the BASIC language module are located in slot 0, which is often not expanded. The SUBROM, disk ROM, and RAM are located in the third expanded slot. Slots 1 and 2 are accessible to the user. On the Daewoo, everything except the disk ROM is located in the expanded slot 0. And the disk ROM itself is located in the non-expanded slot 2! Slot 1 is available to the user, while slot 3 is not expanded and is routed to the back panel of the computer. And while game cartridges worked fine on the Daewoo, the rest of the software often didn't work or worked incorrectly.

There was a third problem as well, but more on that later. So, the idea arose to modify the Daewoo CPC-400 and 400S into something more compatible with various software, remove the glitchy Hangul module and , and add support for the Russian language. The model chosen as a benchmark was the KUVT2, or more precisely, its "teacher" variant based on the Yamaha YIS-805/128R MSX2 computer. In my opinion, this is one of the most ideal

MSX2 computers—both in terms of appearance and configuration. All that was left was to get the Daewoo boards, and I could get started! But that wasn't the end of it... Getting the boards in Europe turned out to be quite a challenge. So I had to search for a long time to find someone who could provide them to me. My longtime colleague Sander, who produces MSX peripherals under the SuperSoniqs brand, was the one who not only helped me get a set of boards but also provided me with a complete Daewoo CPC-400S with a keyboard for testing. Many thanks to him!

Now let's talk about another problem—a very unpleasant one, by the way. Daewoo's engineers made one very serious mistake when designing this computer—they placed the MSX clock batteries onto the video board, which was located above the motherboard and connected to it via cables. The batteries were removable, AA-type. And what do batteries do when they aren't replaced for decades? That's right—they leak. And they leak so badly that they not only destroy the video card sitting above the motherboard, but also manage to damage part of the main board—the electrolyte runs down the cables and causes corrosion. So all the boards I ended up with were severely corroded. One video card was particularly unstable—I had to clean the electrolyte off it, re-solder it, and install additional capacitors. I also replaced all the cables and the battery holder. And on the motherboards, I had to replace some of the IC sockets and cable connectors and clean the green-stained chip pins.

There was another problem—I ended up with a motherboard from a CPC-400S, but the video board was from the 400 model. The thing is, the master crystal oscillator on the 400S is located on the video board, so it's not needed on the motherboard. But my video board from the 400 didn't have that crystal. So I had to figure out how to install the crystal oscillator on the motherboard. This took some work, since I couldn't find any schematics or good photos of the board online. So I reached out to a few MSX enthusiasts living in South Korea, where these Daewoo computers were originally manufactured. This group of Korean users had released clones of the motherboard and video card for the Daewoo 400, so reaching out to them turned out to be a very good idea. I also requested firmware for the programmable logic chips—there are two of them on the Daewoo board—and, as you can imagine, no one had any firmware dumps. I wrote to a man named Peter Yoo, and he kindly provided me with good photos of the new board, the schematic, the pinout, and dumps of both PAL firmware (the U33 and U46 programmable logic chips). According to Peter, the dumps were created by three different Koreans: "기판매니아," "cronosx," and "무건돌." I'd like to express my immense gratitude to all three of them, as well as to Peter, for providing these materials!

And at the very end of the project, when testing was already underway, it turned out that when the disk ROM was moved to slot 0.3, the built-in 3.5-inch floppy drive would take boot priority, and when attempting to boot MSX-DOS2 (Nextor) from a cartridge with a disk drive (e.g., Carnivore2 or 2+) inserted into slot 1, MSX-DOS1 would first boot from the floppy disk inserted into the drive. If no floppy disk is present, MSX-DOS2 will boot from the cartridge, but when attempting to read the directory, the system will report a Disk A: error. Therefore, it was decided to add an alternative file for flashing the U33 programmable logic chip to the repository, which is already ready for release. So, if you plan to use only the built-in floppy drive and game cartridges, write the U33.JED file to the U33 programmable logic chip. If, however, you plan to use cartridges with disk drives, such as Carnivore2 or 2+, then to avoid conflicts with the boot priority, you must write the **U33_DSL2.JED** file to the U33 programmable logic chip. As a result of these steps, the boot priority will be assigned to the disk drive in slot 1, rather than the built-in 3.5-inch floppy drive.

3. Project Goals

With the dumps, a partial schematic, and a set of boards in hand, I could confidently start the project. The official start of the modernization work should be considered mid-July 2025. But, as usual, fate intervened right at the start. To begin with, the board provided for the project turned out not to be fully functional. At first, the crystal oscillator wasn't operating very stably, so I had to select the correct capacitors. The board schematic didn't help here, since the capacitors ratings were listed incorrectly. The oscillator was a modern one, so selecting the capacitors and "tinkering around" took some time. The oscillator's body was grounded and soldered to the board;

for convenience, jumpers were installed to disconnect its pins from the board so that the motherboard could be used with a video board from the 400S. It then turned out that at least two 74LS367 buffers were faulty—they were replaced. After further testing, it turned out that the clock IC couldn't hold data even with charged batteries. The 74LS157 chip was replaced. And as already mentioned, several damaged IC sockets and connectors, onto which battery electrolyte had leaked, were replaced. It was at this point that work began on researching and modifying the computer. Everything that had happened before was just a prelude.

The following goals were set for the project:

- Replace the MSX BIOS and BASIC with Yamaha versions from the YIS-805
- Remove the Hangul module and other Korean ROM modules
- Move the RAM from slot 0.2 to another slot
- Move the disk ROM from slot 2 to another slot, freeing slot 2 completely
- Install the YIS-805 Yamaha SUBROM
- Expand slot 3 and leave it accessible to the user
- Preserve the ability to quickly revert everything to the original configuration
- Make a minimum of cuts and solder a minimum of new wires
- Replace as few chips on the board as possible

At the time, I didn't realize just how problematic it would be. And the name of this problem is the **DW64MX1** chip. Unlike Japanese MSX2 computers, which used the MSX Engine S1985 or S3527 chip, the Koreans installed their own MSX Engine. It's a large 64-pin chip with a 1.78mm pitch, looking very similar to a VDP (video processor). After numerous experiments, a rather bleak picture emerged:

- RAM and the mapper can only be located in slot 0.2
- The mapper is implemented inside the chip and cannot be disabled
- Slot 2 has no physical outputs and is implemented inside the chip
- For some reason, the CS signals are not implemented in slot 2, and it cannot be expanded at all
- The disk ROM does not work outside of slot 2, and the Yamaha's disk ROM does not work on the Daewoo

It then became clear that we would have to work very hard to achieve the project's goals. Replacing the DW64MX1 chip to eliminate the above-mentioned problems would have been a logical solution, since it had already been studied and a prototype for its replacement based on an Altera CPLD had even been created. But that would have gone beyond the scope of our goals. So the focus shifted to analyzing the circuitry to understand how to change the logic of module placement in the Daewoo slots. And judging by the schematic, this is handled precisely by the PAL programmable logic chips we already know, for which we have the original firmware in the form of JED files. Of course, the firmware source code wasn't available, so Maxim Vlasov provided me with invaluable help here. He analyzed and provided me with the logic of both chips in the form of new firmware for these microchips—in source code! Yes, it was now possible to work with this data! Two GAL16V8D chips were taken, and the newly compiled firmware was written to them. Tests using showed that the computer continued to function as before, so we could begin tinkering with the BIOS modules.

4. Analysis and Initial Modification

So, after analyzing the firmware, it became clear that the BIOS modules are stored in two different ROM chips in a specific order. These 64-KB ROMs are located in sockets U38 and U44, both of which are UV-erasable. The layout of the modules is as follows:

Addresses/Slots	0x0000-0x3FFF	0x4000-0x7FFF	0x8000-0xBFFF	0xC000-0xFFFF	ROM Chip
0.0	MSX BIOS	MSX BASIC			U38
0.1		HANGUL1	KANJI CONTROL		U44
0.2	RAM	RAM	RAM	RAM	-
0.3	SUBROM	HANGUL2			U38, U44
2		DISK			U38

In other words, we have 49 KB of Korean language support modules and 64 KB of modules required to boot the MSX2. It should be noted that the MSX BIOS installed in the Daewoo is called X2 and displays a different startup screen, unlike Japanese MSX systems. The disk ROM is also unique, as mentioned earlier. After analyzing the ROM chip dumps, we determined which sections of the ROM images are located in the sub-slots we need. The layout is as follows:

ROM Chip	Module	Address in ROM	Address in slot	Slot number
U38	MSX BIOS	0x0000-0x3FFF	0x0000-0x3FFF	0.0
U38	MSX BASIC	0x4000-0x7FFF	0x4000-0x7FFF	0.0
U38	DISK	0x8000-0xBFFF	0x4000-0x7FFF	2
U38	SUBROM	0xC000–0xFFFF	0x0000-0x3FFF	0.3
U44	HANGUL2	0x0000-0x3FFF	0x4000-0x7FFF	0.3
U44	HANGUL1	0x4000-0x7FFF	0x4000-0x7FFF	0.1
U44	KANJI CONTROL	0xC000-0x7FFF	0x8000-0xBFFF	0.1

The missing 16 KB in U44 ROM at addresses 0x8000-0xBFFF are filled with 0xFF bytes and, apparently, are not mapped anywhere in the system. The module layout is a bit strange, to be honest—in Japanese computers, the disk ROM usually comes last in the 64 KB image. But let's not get sidetracked.

Work on modifying the firmware for the programmable logic chips began as soon as the module addresses and their positions in the ROM images were identified. According to the schematic, the PAL U46 chip supported the Hangul module and influenced the display of text on the screen. Since we removed the Korean modules from the computer, this functionality was also removed from the new U46 firmware, but this was not strictly necessary—we could have simply left the original U46 in place. The following changes were made to the U33 firmware:

- The disk ROM located in the U33 ROM image was moved from slot 2 to slot 0.3
    - If you leave an empty space (filled with 0xFF bytes) in the U33 ROM at addresses 0x8000–0xBFFF, the disk ROM can be loaded into another slot, such as 0.1, but this is not necessary
- ROM U44 is visible only in sub-slot 0.1 at addresses 0x4000-0xBFFF (32 KB)
    - Installing the U44 ROM is now optional
    - We now have 32 KB available in sub-slot 0.1, so we can write 32 KB of a game or utility by placing the first block in the ROM chip at addresses 0x4000–0x7FFF and the second block at addresses 0x8000–0xBFFF

And now it was possible to test the Yamaha BIOS on the Daewoo MSX2. Since the Daewoo has a QWERTY keyboard layout and the YIS-805/128R has a JCUKENG layout, the ROM had to be patched to get the correct layout. I also implemented numeric input without pressing the Shift key, as in the "enhanced" firmware for the Russian KUVT2 computers. The placement of certain characters that didn't match on the keyboards was corrected. A little later, it was discovered that the Daewoo keyboard lacked the backslash (the \ symbol), so it was added by pressing the **"zero"** key **with Shift (0+Shift)**. Since all the BIOS modules were now located on a single chip, it was possible to assemble them all together and finally run the Japanese ROM images (with Russian instead of Korean as the second language) on the Korean hardware. My first video featuring the Daewoo ROM loaded from the KUVT2 dates back to

July 20. But at that time, I had no idea how many additional challenges I would have to overcome before the project was completed...

5. Initial Challenges and Their Solutions

And the difficulties began as soon as the decision of moving the RAM from slot 0.2 to another location was made. It turned out to be impossible to disable the onboard mapper, since it is built into the Korean MSX Engine and there were no jumpers on the board to disable it. Not only that, but if you install a different RAM module in the system—for example, on Andrey Koryakin's RAM expansion board—there will be a conflict between the Daewoo mapper and the mapper on the RAM board, even if you remove all RAM chips from the motherboard. And they would have to be removed, since the built-in mapper cannot be disabled and the onboard RAM will always have priority in the system. That is, the onboard RAM will always be detected first by the BIOS.

Here we come to the most troublesome part of the entire project—the RAM (four 4464-type DRAM chips) is soldered onto the board. And it needs to be removed for the project to turn out as intended. Removing the four chips U18, U19, U20, and U21 from the board is usually no trouble if you have a desoldering pump, a soldering iron, and steady hands. For everyone else (no offense intended), the only option left is to use wire cutters. Cutting out the 4 chips from the board isn't difficult—the main thing is to carefully desolder the cut-out pins afterward using a soldering iron. If you're skilled enough, it's highly recommended to install IC sockets in place of the cut-out or desoldered RAM chips—in case you ever want to return to the original RAM configuration in slot 0.2. And what should you put in place of the removed RAM, you ask? To replace it, external RAM modules were developed from scratch, which are inserted into the rear slot 3 connector. These modules must be manufactured or purchased to successfully complete the project, especially if you're upgrading to MSX2+.

6. External Modules

First, a simple RAM module based on discrete logic was developed, very similar in circuitry to Andrey Karyakin's RAM module, which could be inserted directly into the computer's rear slot and operate in the non-expanded slot 3. At that time, I was still unaware of the mapper conflict. The board allowed the system to boot, but in some cases, the mapper tests with the TestMAP and the TESTRAM utilities returned errors. It became clear that for the RAM to function properly, the BUSDIR signal needed to be controlled, and the first version of the board was unable to do this. So this RAM board was shelved "for later," and the focus shifted to creating an expander for the rear slot 3. Moreover, it soon became clear that the original Daewoo ROMs refused to work with the RAM that is not located in slot 0.2. That is, the system boots up, but the disk BIOS doesn't load anything from the floppy disk. But let's get back to the external modules.

To expand slot 3, a so-called "slot expander" was needed. This isn't a simple device with a couple of logic chips—it requires quite a lot of different logic to control the MSX bus, so the choice was made in favor of an Altera CPLD. It was decided to make a board with a 64KB EPROM socket for slot 3.0 and two 2x25-pin headers for slots 3.1 and 3.2. The plan was for a RAM board to always be inserted into slot 3.2, while slot 3.1 could be used for other modules or cartridges. The ROM for slot 3.0 was intended to contain SUBROM and disk ROM.

A board was also made with a 64KB EPROM panel and a standard MSX cartridge slot. This was for the case where it might be necessary to add something to the system in the form of additional ROM or to install a standard MSX cartridge (the cartridge and ROM cannot be used together). This board can also be used to fully utilize the rear slot 3 as an external expansion slot in certain configurations. So, three different boards were made, and device prototypes were assembled. While the RAM board was more or less straightforward, firmware had to be written in VHDL for the slot expander to function properly.

7. First VHDL Firmware

By that point, I had already studied VHDL a bit and even written some basic firmware for the iBolit2 cartridge that made the LEDs flash in four different ways. I had already gotten the hang of the Quartus software package, could create CPLD projects, set up the correct pinout, and compile firmware. But just a year ago, I was even afraid to look at the source code for the Carnivore2 cartridge firmware. Back then, I didn't understand at all how VHDL described the device's circuit. That kind of "black magic" was too complicated for me. But to finish the project, I had to study the subject after all, and that took some time.

Making LEDs blink is easy. But making sure the MSX slot is expanded, and that various modules work correctly in it—that's much more complicated. And here, once again, I was helped by the esteemed Maxim Vlasov, as well as (I can already see some of you reaching for rotten tomatoes to throw at the author of this document) artificial intelligence (AI), or to be more precise—Google Gemini. I didn't bother ChatGPT or Copilot, but focused on framing the task for Gemini. A properly framed task is the key to success. But, like all beginners, I ended up with a very complex and suboptimal firmware. Using AI, I wrote separate pieces of code for each microchip, and then I connected these pieces according to the slot expander schematic I found online. And it actually worked! Then Maxim Vlasov came to my rescue and created firmware for the CPLD using just a few lines of code. At first, his firmware displayed a black screen, and we spent a long time figuring out why. It turned out that the developers had made a major mistake in the Quartus software package, and the firmware only compiled correctly on the first try; after that, it produced something that didn't work. So before compiling, we always had to clear the project of old data. So on top of everything else, we even found a bug in the Quartus software.

After studying the code for the Maxim firmware, I gained a better understanding of VHDL and how to write optimal code for CPLDs. Since my first RAM module still wasn't working properly in the system, I decided to add BUSDIR signal processing to the firmware for my RAM module, which didn't have that functionality yet. And it worked! The module worked as intended, and there was no longer any conflict with the on-board mapper. This was a great achievement—the problem with the external RAM was finally solved!

After successfully debugging the slot expander firmware, an experiment was conducted to replicate the Russian KUVT2 configuration—that is, to move the SUBROM to slot 3.0 and the disk ROM to slot 0.3. In its new location, SUBROM worked without any issues, but a problem arose with the disk ROM—it flatly refused to work in the expanded slot.

8. New Problems—Disk ROM

After resolving the RAM issue, it was time to move the disk-based ROM to another slot, relocating it from slot 2 to the address 0x4000 in slot 0.3. To do this, the firmware of the U33 programmable logic chip was modified. And that's when we hit a snag. The disk ROM flatly refused to work in the expanded slot! An attempt to replace this disk ROM with another one, for example from a Russian KUVT2 (Yamaha YIS-805/128R), failed. As many as 18 (!) other disk ROMs from all kinds of MSX2 computers—from Brazilian to Japanese models—were tested. The result was zero! If there was one place I didn't expect problems, it was with the disk ROM...

Several days were spent studying the disk ROM's behavior in the openMSX emulator. To start with, the configuration of the already updated Daewoo was recreated in the emulator, the patched modules were connected, as well as the original disk ROM. Just like on a real computer, it worked perfectly in slot 2, but refused to do anything when in slot 0.3. First, I tried patching the disk ROM from the KUVT2 so that the driver would send the correct commands to the disk controller and so that the registers would be located at the correct addresses in the data area. But that didn't help. Then I tried to "extract" the driver from the Daewoo disk ROM and insert it into the disk ROM from the KUVT2. And again, I was unsuccessful...

At some point, out of desperation, I turned to Maxim for advice, and he even started working on adapting the Yamaha disk ROM specifically for Daewoo. But after spending dozens of hours in the emulator, I still managed to solve this problem and patch the original Daewoo disk ROM so that it would work in other slots.

9. The "suicide" of the Disk ROM

The debugger in the latest version of the openMSX emulator has become absolutely fantastic—you can fill the entire screen with windows displaying memory dumps, registers, disassembled code, and so on. The main challenge is to catch the moment when everything “goes haywire,” and then rewind a bit to see what led up to that event. Dozens of hours were spent studying how the disk ROM works and tracking down the source of the problem. In short, it turned out that there was a single check in the code that was supposed to determine which slot the disk ROM was currently operating in. But this check wasn’t being performed fully. As a result, the stack address was corrupted; the stack spilled into the working area and overwrote part of the disk driver located in RAM. Moreover, this overwriting occurred in a loop, so the system simply crashed after a few disk operations, as the working area became clogged with garbage data. This is how the disk ROM’s “self-destruction” occurred.

But as soon as this check was enabled as mandatory, the situation was immediately resolved, the stack corruption stopped, and the disk ROM began working in slot 0.3, as planned. As a result, the disk ROM remained where it was—in the U38 ROM chip—but thanks to the modified logic of the U33 programmable logic chip, it was moved to a new slot, thereby freeing up slot 2.

It is worth noting that you can leave the SUBROM together with the disk ROM in slot 0.3—the system will still work perfectly. You won’t even need a slot expander. But only in one case—if you have a new RAM module that can operate independently when installed in the rear slot 3.

10. New Board Versions

As you may recall, the previous RAM module based on discrete logic did not function properly when inserted into slot 3. It only worked correctly with special modified firmware for the slot expander’s CPLD. But this solution didn’t seem ideal to me—it was still a “workaround.” As a result, a decision was made to create new revisions of all external boards. Also, riding the wave of success, I came up with the idea of making it possible to upgrade the Daewoo to a full-fledged MSX2+ computer.

To upgrade to MSX2+, several things are required: implementation of a single-bit F4 port, an FMPAC sound card, replacement of the video processor with a v9958 model, and replacement of all ROM modules. I’ll write about the upgrade itself later; for now, let’s focus on implementing the F4 port. Where is the most convenient place to implement it? Yes, in the slot expander that runs on a CPLD! All that’s left is to route a few additional signals from the MSX bus to the CPLD, and we’re all set! And that’s exactly why a new revision of the slot expander board was created.

It should be noted that initially I didn’t plan to install sound cards into this slot expander, so I didn’t connect anything to the SNDIN pin (audio input) on the MSX slot. But in the new revision of the board, this connection was made—and for all external sub-slots except slot zero (which only has an EPROM chip). The board with the EPROM socket and standard MSX slot was also modified—there, the SNDIN pin was also connected to the MSX slot, for example, in case the user wants to use an SCC cartridge in the slot expander.

Finally, the RAM module was completely redesigned. It still had 2 megabytes of memory, just like the previous module, but the discrete logic was replaced by the same CPLD used on the slot expansion board. This time, I wrote the CPLD firmware from scratch myself, without any help. Well, maybe I consulted with Gemini a couple of times for optimization. By that time, I was already pretty comfortable writing simple VHDL firmware, so I didn’t even have to ask Maxim for help. The new RAM board now controls the BUSDIR signal on its own, so it doesn’t need the slot expander. It can operate in the non-expanded slot 3 on its own, if necessary. I also added an option to

configure different RAM capacities for this module—512 KB, 1 MB, 1.5 MB, or the full 2 MB. The configuration is selected using four solder jumpers on the back of the board.

11. Per Aspera ad Astra. And Beyond...

So, what was originally planned was successfully implemented, and the glitchy Korean computer turned into something worthy, closely resembling the Yamaha YIS-805. The project took at least six months. Sometimes entire days were spent learning new languages, studying electronic schematics and datasheets, working in KiCAD (highly recommended!), debugging code in the openMSX emulator, and testing on a real computer. Sometimes the project was put on hold while waiting for electronic components or new board versions, or for replies from Maxim or members of the RBSC group. Hundreds of hours of work, hundreds of euros in expenses, sleepless nights, aching eyes. But the result was achieved!

It would seem—that's it, stop! But, as they say, "perfect is the enemy of good". The idea of turning this MSX2 computer into an MSX2+ became the main focus upon the project's completion. Especially since, by this time, I had already converted some computers to MSX2+ and knew what was needed for it. And just then, a stock MSX2 computer with a "random" malfunction turned up—it would freeze for no reason in all games. Extensive troubleshooting yielded nothing, but luckily, this computer had a v9958 video chip. I bought it from the owner and decided to continue the upgrade project.

12. MSX2+ is Always Better than MSX2

There were no issues with the F4 port implementation, and the RAM was already working fine. The slot expander allowed me to use a 64KB ROM in slot 3.0. All the cards were on the table. What else needed to be done for the project? The following:

- Replace the MSX BIOS and BASIC with their MSX2+ versions
- Patch the MSX BIOS to work with Daewoo (adapt the keyboard layout)
- Install a compatible MSX2+ SUBROM and KANJI ROM
- Design and assemble the FMPAC sound card board

I had no problems modifying the BIOS. Since I had previously upgraded my Yamaha 805 to MSX2+, I already had the necessary files on hand. I simply transferred the previously tested file with the modified keyboard layout to it and added the patched disk ROM. The slot expander allowed me to install a 64KB ROM chip with the new SUBROM and KANJI driver—they both take up 49KB and fit perfectly into slot 3.0. The disk ROM didn't get in our way—it stayed in slot 0.3, but when upgrading the Yamaha YIS-805 to MSX2+, the disk ROM had to be moved to another slot.

All that remained was to make an FMPAC board, preferably with stereo sound and an audio socket. The RBSC group has a project for a combined F4/FMPAC board, which is actually where our group's story began. Together with Dmitry Pugachev, I made this board for installation in the KUVT2 (the student version of the Russian YIS-503IIIR). My first board sounded absolutely terrible, but over time I got it working properly. So making a new FMPAC board in the image and likeness of our first project wasn't too difficult. But finding an original YM2413 chip turned out to be much harder! A lot of defective or rebranded chips are coming from China these days. I have about thirty YM2413 chips lying around that sound like a drunken orchestra. But I did manage to find one working chip thanks to Peter Hanraets.

But even here, there were issues. What worked perfectly on the Yamaha computer worked a little differently on the Daewoo. Specifically, the sound volume level was clearly insufficient for the Daewoo. The new FMPAC board included a stereo 3.5mm socket for connecting to an external amplifier, as well as a channel mixer for the SNDIN input of the MSX slot. I had to slightly modify the amplifier circuit right "on the fly" on the old board—

using wires. And once again, Maxim Vlasov's advice came in handy; where would I be without him? The new revision of the board now works perfectly. The sound is clear, without distortion; it's even stereo from the audio jack—the circuit was slightly modified compared to our previous board. The test, of course, was and remains the Sonyc disk game, which perfectly verifies whether the computer has been properly "upgraded" to MSX2+.

And, of course, replacing the video processor with a v9958 was absolutely necessary for a full upgrade to MSX2+. Fortunately, the original video processor is installed in a socket, so it was very easy to remove from the board. Since the pinout of the new video processor differs slightly from the previous one, I had to solder a few wires on the back of the board. And I didn't have to cut any traces. I also had to connect one pin of the video processor to a pin of the Z80 processor via a standard diode, such as a 1N4148. Detailed instructions are provided in a separate document.

13. How to Replicate the Project

So, the project is ready! Anyone who has ever held a soldering iron in his hands can replicate it. There's even an option where you don't need to remove the onboard RAM—you just need to replace a few chips (in this case, some programs still won't work properly due to the RAM's location in slot 0.2). An EPROM programmer, such as the TL866, T48, or similar, is required to write the necessary data to EPROM chips and programmable logic chips. For some upgrade options, you'll need to assemble and purchase additional boards. If you assemble the boards yourself, you will need to program the CPLD using a USB Blaster programmer with the "Quartus II 15.0 Programmer" software package, which can be downloaded from here: https://mirrors.pdp-11.ru/_msx/_carnivore2/quartus/

A separate document named "**Daewoo_CPC400_Upgrade_HowTo.pdf**" provides detailed step-by-step instructions for the modification, complete with photos. Now, let's list the modification options, starting with the simplest and moving on to the more complex ones.

- **[A]** MSX2 minimal upgrade without removing RAM
  - Replacing the main ROM
  - Replacing the programmable logic chip (or both chips)
  - Removing the secondary ROM or (optionally) replacing it with a ROM containing a different program
- **[B]** MSX2 medium upgrade with external RAM
  - Removal of RAM
  - Replacement of the main ROM
  - Replacement of the programmable logic chip (or both chips)
  - Removal of the secondary ROM or (optionally) its replacement with a ROM containing a different program
  - Connecting the RAM module to the rear slot
- **[C]** MSX2 full upgrade with external boards (**recommended**)
  - Removal of RAM
  - Replacing the main ROM
  - Replacing the programmable logic chip (or both chips)
  - Removing the secondary ROM or (optionally) replacing it with a ROM containing a different program
  - Connecting a slot expander to the rear slot
    - Installing the ROM into the slot on the slot expander board
  - Connecting the RAM module to the slot expander
  - Optionally, connecting the FMPAC module to the slot expander
- **[D]** MSX2+ Full Upgrade with External Boards (**Recommended**)
  - RAM removal
  - Replacing the main ROM

- Replacement of the programmable logic chip (or both chips)
- Removal of the secondary ROM or (optionally) its replacement with a ROM containing a different program
- Connecting the slot expander to the rear slot
    - Installing the ROM into the slot on the slot expander board
- Connecting the RAM module to the slot expander
- Connecting the FMPAC module to the slot expander
- Replacing the video processor with a v9958 (installation in the panel)
- Soldering several wires between the video processor pins
- Connecting the video processor to the Z80 processor via a diode

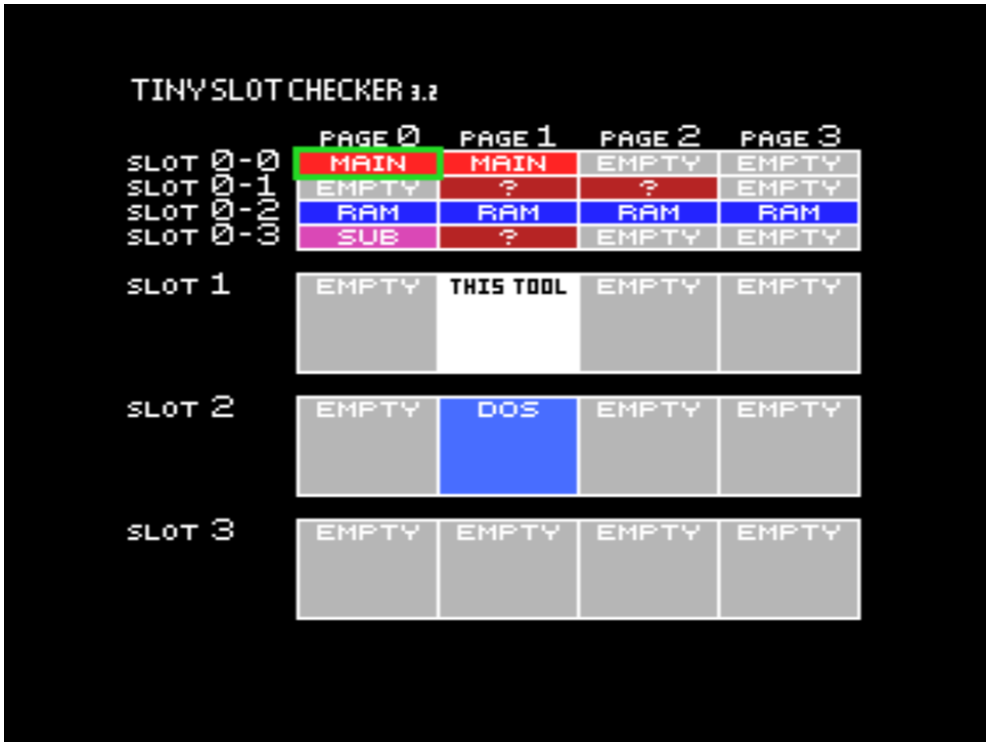
The first option [A] allows you to simply replace two chips, resulting in a more or less compatible computer without Korean modules. However, some programs may not like the RAM in slot 0.2 and may not function properly. On the other hand, you won't need to solder anything. The other options involve removing the onboard RAM and installing additional boards in the rear slot. The project repository contains all the necessary files—ROM and PLD (programmable logic chip) firmware, schematics and Gerber files for the required boards, CPLD firmware for these boards, as well as 3D models of the enclosures for all expansion boards. Also uploaded are the first version of the RAM board based on discrete logic and modified firmware for the slot expander, which prevents this module from conflicting with the onboard Daewoo mapper.

14. New Slot Configurations

Below are screenshots from the Tiny Slot Checker program showing the original Daewoo configuration, as well as the new configurations A, B, C, and D, illustrating the placement of BIOS modules by slot. Note that the new configurations do not show the placement of the optional ROM in slot 0.1 with addresses 0x4000-0xBFFF, nor do they include the possible connection of the FMPAC module in slot 3.1 with address 0x4000. The Tiny Slot Checker program itself had to be placed in slot 1 to create these screenshots.

In the screenshots, MAIN refers to the MSX BIOS and MSX BASIC, SUB refers to SUBROM, RAM refers to RAM, DOS refers to disk ROM, and KANJI refers to the Kanji module (for the MSX2+ configuration). Question marks indicate the presence of unidentified ROM modules—in the original configuration, these are Korean Hangul modules, and in the MSX2+ configuration, this is the second part of the Kanji module.

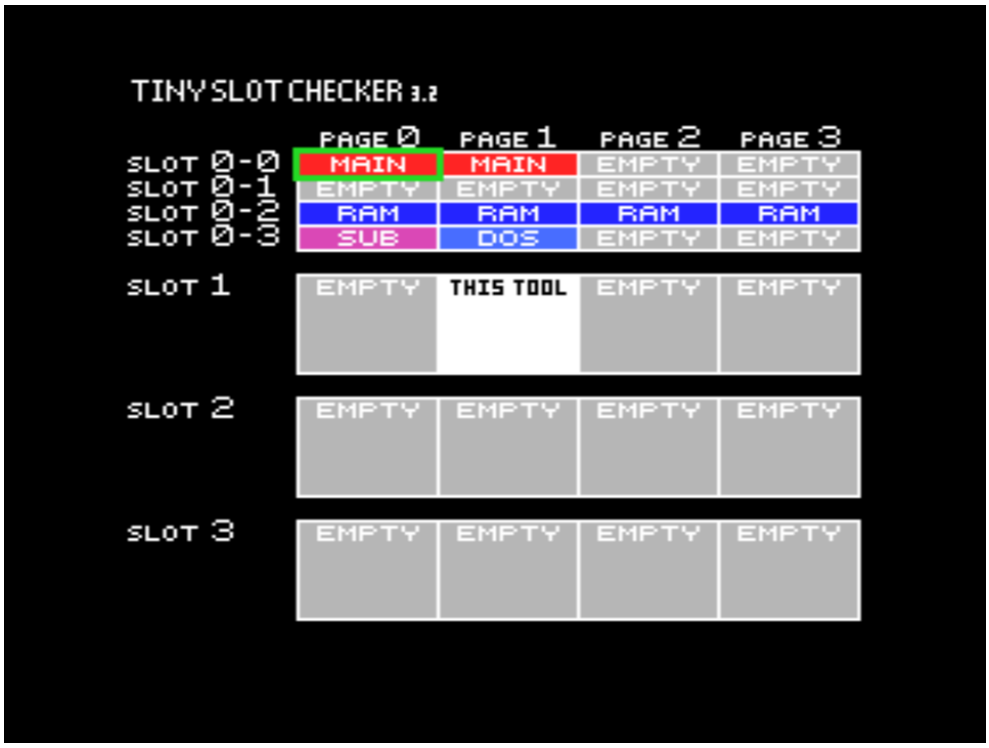
Original configuration:



Screenshot 1. Original configuration

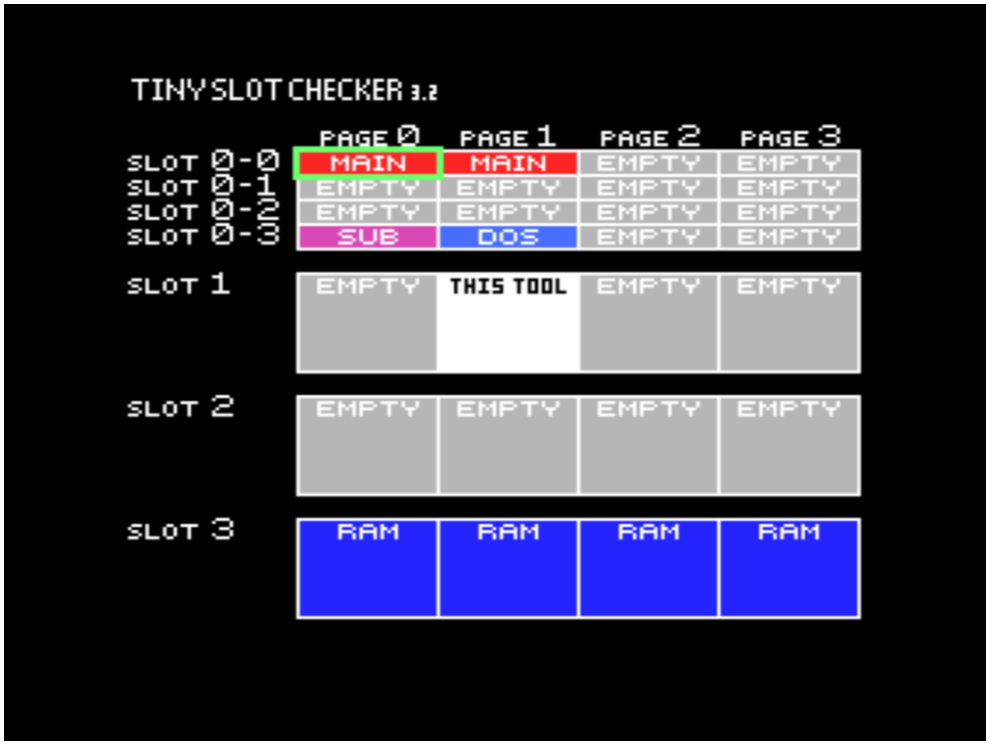
As shown on the screenshot, all modules except the disk ROM in the original configuration are located in expanded slot 0. The disk ROM, however, occupies the entire non-expanded slot 2, which has no physical outputs and is fully emulated by the MSX Engine chip. The main RAM is located in slot 0.2, and this is arguably the main reason for the Daewoo 400's incompatibility with various disk and cassette software. On almost all MSX2 and MSX2+ computers, the main RAM is located in the expanded slot 3 (usually in slot 3.2), where the SubROM and disk ROM (usually in slot 3.0) are also located. Slot 3 is almost always expanded and, on some computers, routed to a 60-pin side connector. For example, on Yamaha computers of the YIS-503, YIS-604, YIS-805, CX5M, CX5MII, and CX11, as well as on the Arabic versions of these computers, the side slot is used for the SFG (OPM) music module. On the "Russian" models YIS-503 and YIS-805, special modules for setting up a local network were installed in this slot.

Configuration A:



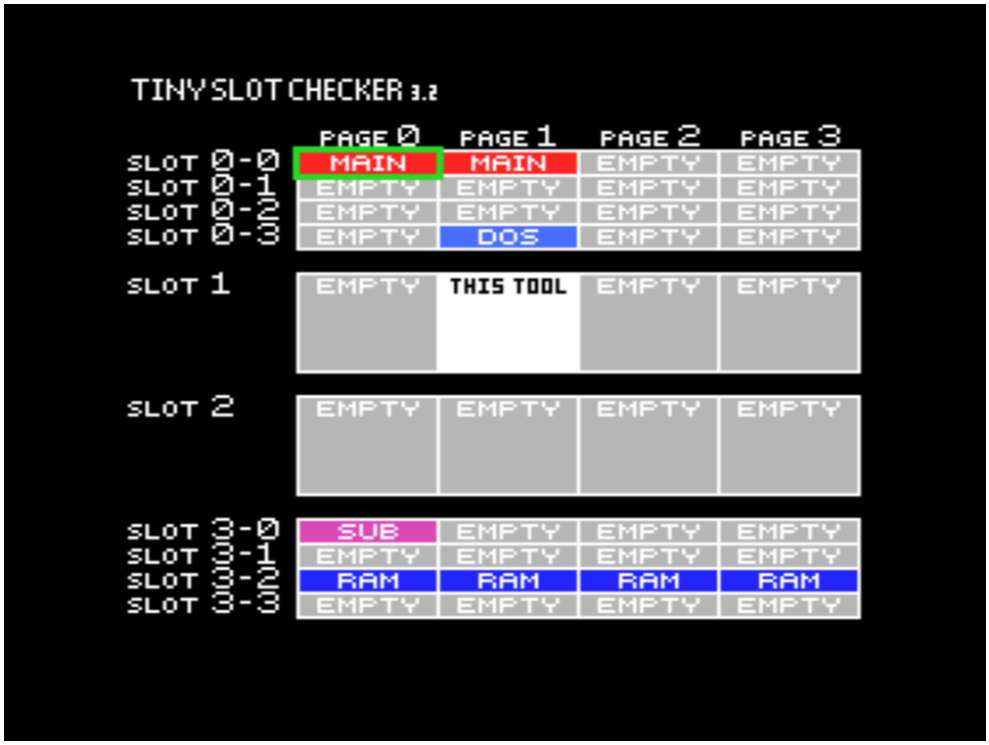
Screenshot 2. Configuration A

Configuration B:



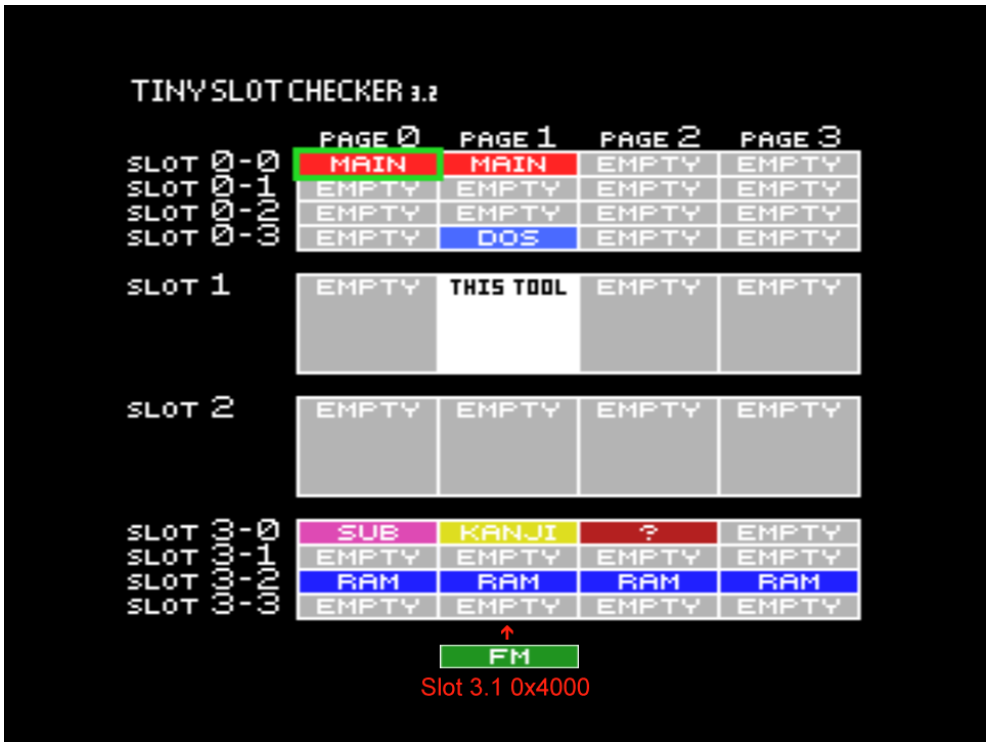
Screenshot 3. Configuration B

Configuration C (recommended):



Screenshot 4. Configuration C

Configuration D (recommended):

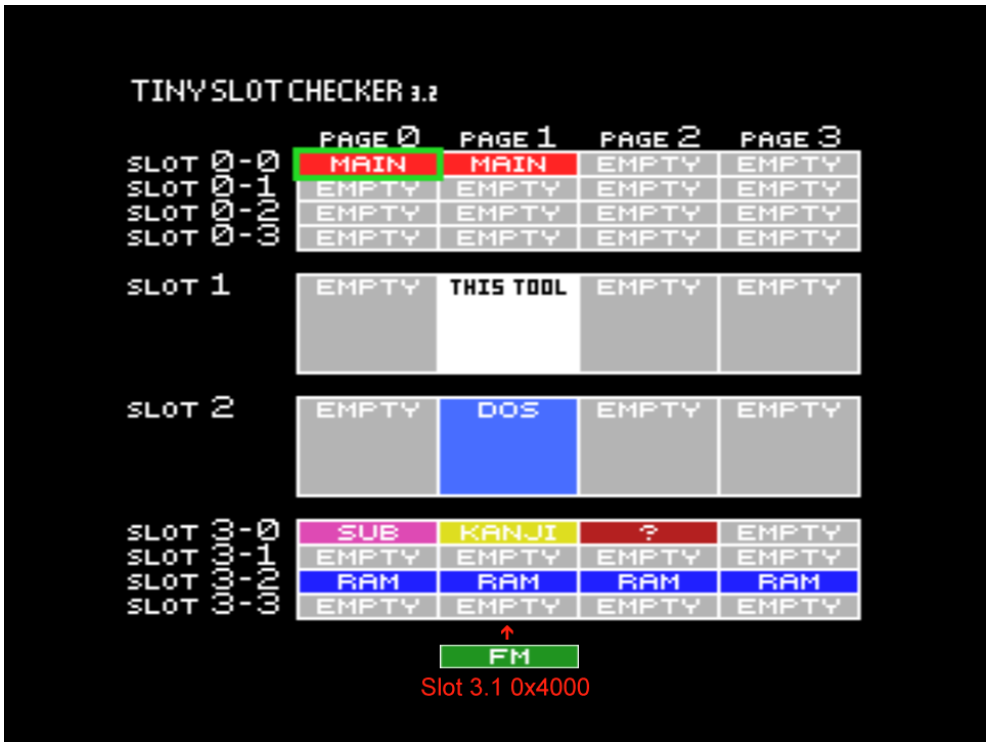


Screenshot 5. Configuration D

As shown on the screenshots of the new configurations, the disk ROM is always located in slot 0.3 at address 0x4000. This is because the U33 programmable logic chip only receives signals from slots 0.1, 0.3, and 2, but not from slot 3. Theoretically, it is possible to fully replicate the configuration of the Yamaha YIS- 805/128R and

place the disk ROM in slot 3.0 at address 0x4000. However, this would require additional modifications to the board (a signal from slot 3.0 would need to be routed), different firmware for the U33 programmable logic chip, and a different arrangement of modules within the ROM chip for the slot expander. This configuration is not part of the plans for this upgrade project, but may be implemented later.

As noted above, when using the U33.JED file to flash the U33 programmable logic chip, issues may arise when using other disk devices in slot 1, such as Carnivore2 and 2+ multi-cartridges. In this case, it is recommended to use the alternative U33_DSL2.JED firmware for the programmable logic chip. This firmware returns the disk ROM to slot 2, so that the disk drive in slot 1 will have boot priority. Here is an example of an MSX2+ configuration with a disk ROM in the non-expanded slot 2:



Screenshot 6. Alternative configuration

15. IMPORTANT INFORMATION!

- All work must be performed with the computer powered off
- It is extremely important to clean corrosion off the board and replace rotted wires and components
- It is recommended to install IC sockets in place of desoldered or cut-out RAM chips
- Cut-out RAM chip's pins must be removed one by one using a soldering iron
- The U38 and U44 ROM chips can be replaced with any UV-erasable EPROM chips of the 27C512 type, or with electrically erasable Winbond W27C512 EEPROM chips. The same 64 KB EPROM chips are installed on external boards (slot expander, board with a standard MSX slot)
- The PAL16L8A programmable logic chips can be replaced with equivalents, such as the Lattice GAL16V8D, or similar chips from Microchip Technology (ATF16V8B)
- A standard 1N4148 diode is used to connect the Z80 processor and the v9958 video processor (video: https://youtu.be/dz-oaFYrFdg?t=1072)
- Jumpers on the back of the board are soldered between five different pins of the v9958 video processor (video: https://youtu.be/dz-oaFYrFdg?t=1015)
- If an EPROM chip is installed on the board with the MSX slot, you cannot insert a cartridge into that slot, otherwise there may be a conflict between the two different ROMs

- The 3D-printed enclosures for all external devices are designed to secure the boards internally with very short 2mm-diameter screws. Do not neglect this. The enclosure halves should be joined with 2x12mm screws. Tighten the screws carefully so as not to damage the plastic
- When soldering the 50-pin female connector to all external boards, make sure the bottom surface of the connector is **on the same level** with the top surface of the circuit board! Otherwise, the assembled device will not fit into the enclosure.

16. Project Repository

This project's repository is located in the main RBSC group repository on GitHub. Project documentation may be updated periodically, so be sure to check that you have downloaded the latest available version.

Permanent link to this repository: https:// github. com/ RBSC/ Daewoo_ CPC400_ Upgrade

17. Contact Information

You can send an email to the RBSC (Russian Bear Service Crew) development team—comprising Tnt23, Wierzbowsky, Pyhesty, Ptero, GreyWolf, SuperMax, VWarlock, ALSP, and DJS3000—at this email address (we speak English, German, and Russian): info@rbsc.su

You can contact the author of this document at this email address: wierzbowsky@rbsc.su

You can visit the RBSC group's website via these links:

https://rbsc.su/
https://rbsc.su/ru

You can visit the RBSC group's repository on GitHub via this link:

https://github.com/rbsc

You can visit the RBSC group's 3D model repository via this link:

https://www.thingiverse.com/groups/rbsc/things

-= ! MSX FOREVER ! =-